Problema 1:

- `Number n = 5;` → se aplica constructorul de inițializare cu o valoare de tip integer (`Number(int n) { value = 1 << n; }`) → prin urmare n.value = 1 << 5 (2^5) = 32
- `n = (int)(n = 2);` → în ordine se efectuează următoarele operații:
  - `n = 2` → se aplica operatorul de asignare (`Number& operator = (int val) { value += val; return *this; }`) → n.value devine 32 + 2 = 34 → apoi se returnează o referință către obiectul curent
  - `(int)(referinta)` → se aplica operatorul de castare la int (`operator int() { return (value++); }`) care returnează n.value (adică 34) dar fiind post-incrementare, îl și creste după returnare pe n.value la 35
  - Deci expresia (`(int)(n = 2);`) se evalueaza la 34, iar n.value este 35
  - Asignarea finala n = 34 fa apela din nou operatorul de asignare și aduna la n.value (care este 35 din pasul precedent) valoarea 34 → în final rezultatul va fi ca n.value este 69
- `cout << (int)n << endl;` → se aplica operatorul de castare, se returnează 69 și n.value va deveni 70
- Programul afișează 69. Răspuns corect **D**. Se punctează:
  - **1 pct** pentru calculul lui n după apelul de constructor (n.value = 32),
  - **2 pct** pentru calculul (`n = (int)(n = 2);`) ⇒ adică tot 69
  - **1 pct** pentru afișarea finală (69).

Nu se iau în considerare nici un alt răspuns. Se punctează răspunsurile doar dacă sunt precedate de explicația corespunzătoare.

Problema 2:

- `cout << (int)(f(5, 3) + f(5.0, 3) + f(5, 3.0)) << endl;` → se evalueaza:
  - f(5,3) → 5 si 3 sunt int-uri deci se face împărțire la **int** → f(5,3) = 5/3 = 1
  - f(5.0,3) → se face un lambda cu **double** si **int**, se face împartire double la int → 5.0/3 = 1.66
  - f(5,3.0) → se face un lambda cu **int** si **double**, se face impartire int la double → 5/3.0 = 1.66
  - Se aduna 1+1.66+1.66 = 4.33 și se castează la int → 4
- Programul afișează 4. Răspuns corect **E**. Nu se iau în considerare nici un alt răspuns. Se punctează răspunsul 4 doar dacă e precedat de explicația corespunzătoare.

Problema 3:

Mai întâi rezolvăm MACRO-urile. După înlocuirea lor codul va arata în felul următor:

```
#pragma pack(4)
class A { int x[1]; virtual void f_A() {} }
class B { int x[3]; virtual void f_B() {} }
class C : public A { int x[1]; virtual void f_C() {} };
class D : public B, public A { int x[1]; virtual void f_D() {} };
class E : public C, public D { int x[3]; virtual void f_E() {} };
```

Directiva **pragma pack** spune ca fiecare structură se va alinia la 4 octeţi (chiar dacă suntem pe 64 de biţi unde un pointer are 8 octeţi) ⇒ traducerea e ca fiecare structura / clasa trebuie sa aibă o dimensiune divizibila cu 4. Avem:

- Clasa A ⇒ un int (4 octeţi), un vfptr (8 octeţi) ⇒ **12 octeţi**
- Clasa B ⇒ trei int-uri (4x3 = 12 octeţi), un vfptr (8 octeţi) ⇒ **20 octeţi**
- Clasa C ⇒ are un **vfptr** (pentru f_C() şi pentru f_A() ⇒ adică 8 octeţi) + un int din clasa C (4 octeţi) + un int din clasa A (4 octeţi) ⇒ 8 + 4 + 4 = **16 octeţi**
- Clasa D ⇒ derivare multiplă de 2 **vfptr-uri** (8 x 2 = 16 octeţi) + un int din clasa D (4 octeţi) + trei int-uri din clasa B (4 x 3 = 12 octeţi) + un int din clasa A (4 octeţi) = 16 + 4 + 12 + 4 = **36 octeţi**
- Clasa E ⇒ derivare multiplă de 2 **vfptr-uri** (8 x 2 = 16 octeţi) + 2 x int din clasa C (unul din clasa C şi unul din C::A) = 2 x 4 = 8 octeţi , plus 5 int-uri din clasa D (1 x int din D, 3 x int din D::B, 1 x int din D::A) = 5 x 4 = 20 octeţi, şi trei int-uri ale sale (3 x 4 = 12 octeţi). În plus derivarea din D implica şi un al doilea vfptr (deci 8 octeţi).Deci in total = 16 + 8 + 20 + 12 + 8 = **64 octeţi**

Se punctează:

- **1 pct** pentru calcularea corectă a size-ului lui A şi lui B
- **1 pct** pentru calcularea corectă a size-ului lui C
- **1 pct** pentru calcularea corectă a size-ului lui D
- **1 pct** pentru calcularea corectă a size-ului lui E

Problema 4:

Iteratorul este scris corect (are cele trei operatori necesari: ++, * si !=).
Metodele **begin()** şi **end()** din clasa Data sunt descrise în felul următor:

```
Iterator begin() { return { a,1 }; }
Iterator end() { return { a,10 }; }
```

Deci returnează o structura de tipul Iterator cu Iterator::a = vectorul a din Clasa Data şi Iterator::poz cu valoarea 1 , respectiv 10 (pentru end()).
Operatorul ++ din clasa Iterator îl creste pe poz din 2 în 2 (`poz = (poz + 2) % 10;`) dar pentru ca se face modulo 10, practic poz dacă porneşte de la valoarea 1 va avea următoarele posibile valori: 1,3,5,7,9. Când ajunge la 9, la următoarea iteraţie (9+2) % 10 = 1 şi se reia ciclul de la capăt. Cum bucla for se efectuează pana când se ajunge la iteratorul rezultat din execuţia metodei **end** deci pana la valoarea 10, rezultatul va fi o bucla infinita în care se vor afişa în continuu 2,4,6,8,10 (valorile din vectorul a corespunzătoare indecşilor 1,3,5,7 şi 9).

Răspuns corect **D**. Se dau 2 pct dacă se precizează ca a intrat în bucla infinita şi încă 2 puncte dacă se precizează şi ce se va afişă în acea bucla (2,4,6,8,10). Nu se iau în considerare nici un alt răspuns. Se punctează răspunsul doar dacă e precedat de explicaţia corespunzătoare.

Problema 5:

Clasa **Number** are operatorul de decrementare descris prin doua funcţii (post şi pre-fixare).

```
[POST]    Number& operator--(int n) { value <<= 1; return *this; }
[PRE ]    Number& operator--() { value++; return *this; }
```

Apelul iniţial al constructorului (`Number n(17);`) va pune în n.value pe (17 >> 1) << 1 = 16 (împărţire la 2 urmata de înmulţire la 2 pe int-uri).
Deci ⇒ codul care se rulează apoi (--((--(n--))--);) il putem scrie ca fiind: POST, PRE, POST, PRE
Rezultatul va fi ⇒ (((16 x 2) + 1) x 2) + 1 = 67

Ultima secventa (`cout << 1.0 + n << endl;`) încearcă sa adune pe 1.0 (un double) cu un Number. Nu exista nici un operator care poate face asa ceva, dar exista castare a unui Number la double (`operator double() { return (double)(value << 1); }`). Deci se aplica castarea la double pe Number (se returnează 67 x 2 = 134 ⇒ convertit la double = 134.0) şi apoi se aduna 1.0.

Programul afişează 135. Răspuns corect **D**. Se punctează 3 puncte dacă se calculează corect expresia --((--(n--))--) ⇒ adică se ajunge la valoarea 67 şi încă 1 punct pentru partea de afişare. Nu se iau în considerare nici un alt răspuns. Se punctează răspunsul 135 sau 135.0 doar dacă e precedat de explicaţia corespunzătoare.

Problema 6:

Mai întâi rezolvăm template-urile. După înlocuirea lor codul va arata în felul următor:

```
class Number_int_float
{
   int value;
public:
   Number_int_float(const int &val):value(val) {}
   operator float () {return (float)(value / 2); }
   int operator+(const int &val) {
       return value + val; }
};
```

```
class Number_float_int
{
  float value;
public:
  Number_float_int(const float &val):value(val) {}
  operator int() { return (int)(value / 2); }
  float operator+(const float &val) {
        return value + val; }
};
```

Deci codul din main îl putem transcrie în felul următor:

```
int main() {
	Number_int_float n1(10);
	Number_float_int n2(3.0);
	std::cout << n1 + n2 << "," << n2 + n1 << "," << n1 - n2 << std::endl;
	return 0;
}
```

Deci - executia pe rand face:

- n1+n2 ⇒ nu exista operator direct de adunare intre cele doua tipuri, dar n2 de tipul **Number_float_int** are un operator de cast la int, iar n1 de tipul **Number_int_float** are operator de adunare cu un int. In acest caz, compilatorul traduce n1 + n2 in **n1.operator+((int)n2)**. Prin urmare ⇒ n1+n2 va afişă 10 + (int)(3.0/2) = 11
- Similar şi pentru n2+n1 ⇒ se va afişa 3.0 + (float)(10/2) = 8
- Pentru ultimul caz, n1-n2 compilatorul încearcă următoarele lucruri:
  - Verifica dacă exista o funcţie de scădere intre cele doua (nu exista)
  - Exista în schimb o promovare la fiecare tip (**DOAR UNA)** posibila - mai exact n1 se poate conversie la un float şi n2 la int. Cum exista o singura conversie ⇒ nu avem un caz de ambiguitate ⇒ se face conversie şi încearcă scăderea. Cum “float minus int” e posibil, compilatorul va calcula acest lucru , adică ((float)n1 - (int)n2) ⇒ (float)(10/2) - (int)(3.0/2) ⇒ 5.0 - 1 = 4.0

Răspuns corect **D**. Se punctează:

- **1 pct** pentru calcularea corectă a rezultatului n1 + n2
- **1 pct** pentru calcularea corectă a rezultatului n2 + n1
- **2 pct** pentru calcularea corectă a rezultatului n1 - n2

Problema 7:

Răspuns:

a) Un tip de date este format dintr-o mulţime de entităţi de tip dată şi de o colecţie de operaţii peste acestea. O clasă defineşte un tip astfel: entităţile de tip dată sunt descrise de către stările obiectelor (instanţelor) clasei, iar operaţiile sunt descrise de metodele clasei. Deoarece un utilizator are acces doar la membrii publici, doar aceştia vor fi consideraţi în definiţia tipului descris de clasă.

b) Principiul substituirii al lui Liskov precizează când tipul definit de o clasă derivată (copil) este subtip al tipului definit de clasa de bază (părinte). Poate fi formulata astfel:
Clasa A este un subtip al clasei B (în sensul definiţiei de mai sus) atunci când o referinţă la o instanţă A poate apare în orice punct al programului în care o instanţă B este aşteptată, fără a altera corectitudinea programului. În consecinţă, metodele suprascrise în A nu trebuie să altereze comportamentul instanţelor A atunci când sunt utilizate pe rol de instanţe B. Această proprietate este descrisă mai precis utilizând conceptul de “design by contract”: 1) precondiţia unei metode suprascrise în clasa copil A nu trebuie să fie mai tare decât cea din clasa părinte B; şi 2) post-condiţia unei metode suprascrise în A nu trebuie să fie mai slabă decât cea din B.
Se punctează:
   - **1 pct** pentru înţelegerea noţiunii de tip
   - **1 pct** pentru descrierea tipului definit de o clasă
   - **2 pct** pentru descrierea corectă a principiului substituirii (ca mai sus sau echivalent)